



Fully hardware based WFQ architecture for high-speed QoS packet scheduling

Kieran McLaughlin*, Dwayne Burns, Ciaran Toal, Colm McKillen, Sakir Sezer

Queen's University Belfast, ECIT, Queen's Road, Queen's Island, Belfast BT3 9DT, UK

ARTICLE INFO

Article history:

Received 25 May 2010

Received in revised form

25 November 2010

Accepted 14 January 2011

Available online 28 January 2011

Keywords:

Fair queuing

WFQ

Packet scheduling

QoS

Traffic management

Shared buffer

FPGA

ABSTRACT

A full hardware implementation of a Weighted Fair Queuing (WFQ) packet scheduler is proposed. The circuit architecture presented has been implemented using Altera Stratix II FPGA technology, utilizing Reduced Latency DRAM (RLDRAM) II and Quad Data Rate (QDR) II SRAM memory components. The circuit can provide fine granularity Quality of Service (QoS) support at a line throughput rate of 12.8 Gb/s in its current implementation. The authors suggest that, due to the flexible and scalable modular circuit design approach used, the current circuit architecture can be targeted for a full ASIC implementation to deliver 50 Gb/s throughput. The circuit itself comprises three main components; a WFQ algorithm computation circuit, a tag/time-stamp sort and retrieval circuit, and a high throughput shared buffer. The circuit targets the support of emerging wireline and wireless network nodes that focus on Service Level Agreements (SLA's) and Quality of Experience.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

Emerging Internet services utilizing wireline and wireless platforms increasingly require greater bandwidth, lower end-to-end propagation delays and improved Quality of Service (QoS) guarantees. Applications such as Voice over Internet Protocol (VoIP), streaming audio, video and other real-time applications have specific bandwidth and propagation delay requirements. These demands create a bottleneck in the routers that form the infrastructure of the Internet, as they must handle ever increasing amounts of data.

Future demands for lower end-to-end propagation delays, greater bandwidth and dependable QoS guarantees cannot be suitably achieved with the ubiquitous best effort model, where packets from different types of traffic are treated without regard for priority needs. Real-time services require dependable performance bounds for delay, jitter and packet loss, which the best effort model cannot guarantee. However, voice, video and other streaming media represent a class of service that is extremely important for the future profitability of Internet Services Providers (ISPs).

Wireless operators in particular are experiencing an explosion in the bandwidth consumed by ever increasingly sophisticated

mobile communication devices. Consequently, QoS for differentiated traffic classes and flows has become a major issue for emerging Long Term Evolution (LTE) wireless technology [1,2], where the provisioning of Service Level Agreements (SLA) and fine granularity QoS guarantees on a per-flow basis are vital for future operator business models, that plan to provide tiered SLAs to customers and guaranteed Quality of Experience for content providers [3].

Fairly arbitrating bandwidth and managing the QoS requirements between a large number of users, each potentially generating many different sessions and flows, is a challenging endeavor at the high throughput rates of modern networks. The processing power required to achieve this at the networks nodes is immense, consequently software based solutions are incapable of supporting complex functionality. Therefore, offloading such processor intensive features to hardware is vital.

1.1. Outline of research

This paper presents the derivation of a Weighted Fair Queuing (WFQ) packet scheduler that has been fully implemented and verified as part of a commercial intellectual property core. It supports traffic management for QoS deployment at 12.8 Gb/s line-speeds, using FPGA technology, and combines a number of key research projects targeting WFQ algorithm processing handled entirely in hardware, WFQ associated tag sorting and lookup in custom accelerated hardware, and a scalable shared

* Corresponding author.

E-mail address: kieran.mclaughlin@ee.qub.ac.uk (K. McLaughlin).

buffer memory capable of managing data payload storage and retrieval for multi-gigabit line-speeds. The circuit has been implemented using Altera's traffic management development board (Fig. 1), which utilizes Stratix II FPGAs, with access to a number of onboard resources such as RLDII and QDRII memory components.

This work proves the feasibility of the circuit for deployment using more advanced FPGA technology or ASIC, targeting packet throughput rates of up to 50 Gb/s. The traffic manager is suitable for core and access deployment in traditional wireline applications, and also in future LTE wireless networks, where it can be used to enable support for SLA's and Quality of Experience.

First the design and implementation of a WFQ computation circuit for high performance Internet Protocol (IP) packet scheduling has been presented. It utilizes a highly pipelined and customized parallel processing architecture, enabling extremely fast finishing tag computation to be achieved. The implementation has a relatively small hardware cost. The embedded memory used in this implementation was dimensioned to support up to 8000 sessions, without the use of external memory. However it can support up to 64,000 different sessions without modification if sufficient embedded or external memory for session parameter storage can be accommodated. Further extension of the lookup table hierarchy permits the support of 8 million different sessions or individual IP flows.

Secondly a novel finishing tag lookup table architecture has been derived that has a number of key advantages over the previous state of the art in this area. In particular this includes a fixed and predictable lookup time and the ability to guarantee that the lowest tag value will always be found. Furthermore, having analyzed a range of viable alternative options, including standard software arrangements, it has been shown that the multi-bit tree based design offers the optimum solution available, both in terms of speed and performance. The impact of this architecture is that it is scalable in terms of the number of tags, sessions and packets supported, delivering high performance with low latency. This circuit can also be deployed for algorithms other than WFQ, which also require high-speed finishing tag sorting and lookup.

Finally the design and optimization of high performance shared buffer architecture has been presented, which uses standard FPGA and RLDII technology to achieve throughput rates comparable with standard cell (ASIC) technology. Current dynamic RAM technology is optimized for cache based micro-processor systems that achieve maximum memory bandwidth access only in burst mode. Utilizing DRAM technology for shared

buffer design requires fine tuning of the access mode to optimize it for the capabilities of the memory technology. The study has shown that operating a shared buffer architecture using FPGA technology is possible even beyond the 12.8 Gb/s achieved using RLDII memory technology.

The overall performance of the WFQ circuit is limited by the technology available on the development board, particularly the memory bus between the FPGA and the RLDII. In order to support a packet throughput rate of 12.8 Gb/s, the overall memory bandwidth required is $2N$ (25.6 Gb/s) and the overall systems bandwidth at the control device (FPGA), excluding packet and flow descriptor memory access, is $4N$, i.e. over 50 Gb/s for 12.8 Gb/s shared buffer.

The remainder of the paper will begin by discussing the background of QoS and various scheduling approaches. Following this the WFQ computation circuit, the tag sorting circuit and the shared packet buffer are derived. Finally the implementation of the FPGA based prototype is presented. Further work discussed in the conclusion provides evidence that state of the art silicon technology can be used to support packet buffering, and WFQ itself, at up to 50 Gb/s.

2. Background

2.1. Quality of service

Delivering QoS means guaranteeing given parameters within certain bounds for connections made over a network [4]. QoS can be applied differently to connections or users, as well as to different types of traffic and data flows. The parameters involved in QoS can be classified as delay, jitter and packet loss.

Delay is inherently unpredictable under best-effort, except to say that if a network is heavily loaded, the amount of delay is likely to increase since packets can be held up in long queues at busy network nodes. Jitter occurs when packets in a data stream reach their destination with different time delays. For streaming services such as VoIP, video conferencing and video on demand, this means video and audio can be intermittent. Jitter compensation is achieved with large de-jitter buffers, which introduce additional end-to-end delays, dependent on the maximum jitter value. Finally, when a network is congested, buffers can become full and packets may be dropped as a result, causing delays and jitter as packets must be requested again and retransmitted.

The circuit presented in this paper provides guaranteed levels of QoS using packet scheduling. The term "scheduling" encompasses a number of policies on which decisions are made when processing packets arriving and departing from a router. At a router, a number of "sessions" (or "flows") exist. These are streams of packets moving between nodes connected on a network. A number of different scheduling techniques exist for QoS and traffic management. Their main objective is to treat different traffic classes or flows of packets with a variable degree of priority in order to provide performance guarantees for a range of different traffic types and profiles. Most techniques aim to emulate the ideal scheduling capabilities of Generalized Processor Sharing (GPS), which is a fluid model, where packets are organized into logical queues with an infinitesimally small amount of data serviced from each non-empty queue in turn. This achieves fair allocation of bandwidth but is not practical since it does not treat packets as complete entities. It does however provide a theoretical standard against which the performance of practical scheduling policies is benchmarked.

Fig. 2 shows how each flow in GPS is allocated a weight proportional to the amount of the total available bandwidth they have been allocated. For example, flow D is allocated twice as

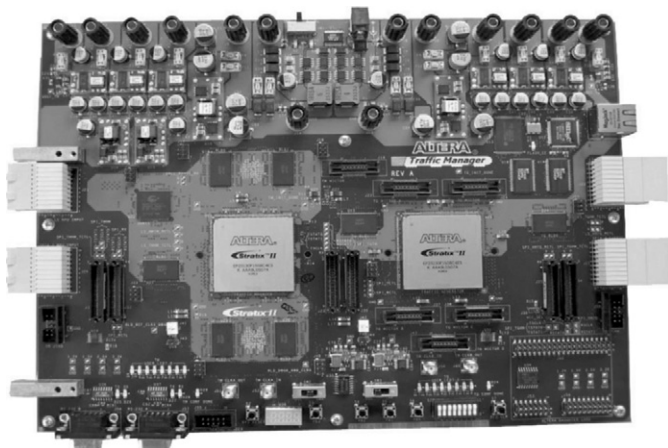


Fig. 1. Altera Traffic Manager Development Board, used to implement the WFQ scheduling traffic manager.

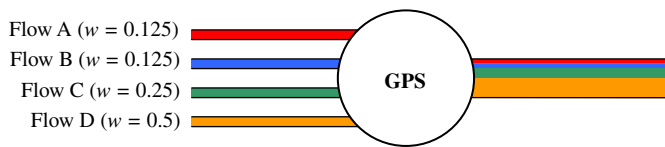


Fig. 2. Generalized Processor Sharing (GPS) model, where w =relative weight allocated to flow.

much bandwidth as flow C. For any time interval, each flow will have been served equally according to the weights allocated to them. Because IP packets are different lengths means that in practice all flows cannot be served equally as intended, since the scheduler must finish serving each individual packet before it moves on to the next one.

Numerous methods have been devised to function as close as possible to GPS, without assuming packets are infinitesimally small, such as Weighted Round Robin (WRR) [5] and Deficit Round Robin (DRR) [6]. More advanced schemes such as Weighted Fair Queuing (WFQ) have also been developed [7], where a virtual time tracks simulated GPS, allowing a worst case end-to-end queuing delay to be provided for guaranteed service connections. These algorithms are all conventionally implemented in software in commercial routers. The different families of algorithms are discussed in the following sections.

2.2. Round Robin scheduling

Round robin is a time-share approach where a “quantum” of data is processed from each input in turn to achieve fairness. A “round” is complete when all connections have been serviced once. WRR allocates weight to flows in order to apportion priority. To calculate normalised weights, the average packet size must be known, which is impractical for IP connections, and with a large number of connections, or small weights, WRR can be unfair over long periods.

DRR processes variable size packets without knowing their mean size, serving each flow only when a “deficit counter” for each queue is greater than the size of the packet at its head. A variation of DRR called Modified Deficit Round Robin (MDRR) adds prioritization in an effort to provide a minimum delay for differentiated services. Cisco is known to have implemented MDRR in some routers, for example to allow VoIP to be prioritized. VoIP packets are identified using the Types of Service (TOS) bits in the packet header. MDRR gives flows identified as VoIP precedence over other packets, either by always serving these packets while letting all others wait, or by skewing the round robin so that a VoIP flow is visited more often than other flows.

The principal drawback for round robin approaches is that bounded delays cannot be provided effectively. Some research has shown this is possible for fixed packet sizes based on modifying a classic round robin approach [8]. The authors suggest the scheme could be extended to variable packet sizes by using a credit based system; however, no analysis or implementation of this has been carried out thus far. Although round robin and its derivatives work well for protocols that use fixed packet sizes, such as Asynchronous Transfer Mode (ATM), they have proven to be unable to guarantee delay bounds for variable sized packets, and are unsuitable for providing full QoS for networks such as the Internet.

2.3. Fair queuing scheduling

A number of more advanced algorithms known as fair queuing scheduling act more fairly and more closely to GPS than round robin approaches. WFQ [7] and Packet-by-packet Generalized

Processor Sharing (PGPS) [9], use a “virtual time” to track the progress of simulated GPS, allowing a worst case end-to-end queuing delay to be guaranteed for connections. These are effectively the same algorithm, developed independently, and were the first to become widely adopted in industry, for example by Cisco.

2.4. Weighted Fair Queuing

WFQ can be described as a rate based flow control strategy, where a traffic source is statistically characterized by rate, burstiness, etc. Worst case or average delays may also be specified, with the aim of providing guarantees on throughput and worst case delay. WFQ operates a simulation of GPS in parallel using a virtual time in order to decide which packet will be served next. This is determined as the next packet that would be served under GPS if no further packets were received after time τ , assuming that the server becomes free at time τ .

For each packet the WFQ simulation of a virtual time is used to allocate a “start tag” and also a “finishing tag”, which is the time the packet would leave the scheduler if it were operating GPS. Each packet is stored along with its designated finishing tag and when the scheduler output is free, the packet with the smallest finishing tag is transmitted. WFQ outperforms round robin approaches because it approximates GPS within one packet transmission time regardless of the arrival patterns and handles variable packet lengths in a more systematic approach.

2.5. Other fair queuing algorithms

Class-Based Weighted Fair Queuing (CBWFQ) is a limited implementation of WFQ that acts on user defined classes of traffic, rather than on a per-flow basis. CBWFQ has been implemented for example on Cisco's 2600, 3600 and 7200 routers, where it supports a maximum of 64 classes of traffic.

Frame-based Fair Queuing (FFQ) and Starting Potential-based Fair Queuing (SPFQ) are both Rate Proportional Service (RPS) scheduling algorithms [10]. FFQ recalibrates periodically, where fairness depends on the frame size. In comparison, SPFQ recalibrates at the end of each packet transmission, which requires more state information to be stored, making it more complex. Self Clocked Fair Queuing [11] is derived from WFQ. It is less complex but has inefficient latency tuning characteristics and can have a large delay bound. Other similar algorithms that are less complex than WFQ, but also less fair, include Credit Based Fair Queuing (CBFQ) [12], which is similar in operation to DRR.

Worst Case Fair Weighted Fair Queuing (WF²Q) is more complex than WFQ but it has better fairness [13]. WFQ cannot fall behind GPS by more than one maximum size packet (the delay bound), but it can end up ahead of GPS. In WF²Q the algorithm is bounded by one maximum packet size behind or ahead of GPS, meaning its worst case fairness is better. However, it is more complex in terms of updating the virtual clock and, like WFQ, requires sorting finishing tags at the output. WF²Q+ possesses the properties of WF²Q, but has a less complex procedure for updating the virtual clock [14]. However, as well as sorting at the output (virtual finishing time) it also requires virtual starting times to be sorted to update the virtual clock, i.e. two complex tag sorting operations per packet.

2.6. Summary

For variable sized IP packets, it has been shown that fair queuing algorithms offer a number of advantages over alternative algorithms in terms of delivering QoS; such as guaranteeing

minimum bandwidth, sharing excess bandwidth fairly and providing guaranteed delay bounds. Despite these clear advantages, a major reason often cited for using methods other than fair queuing is that it involves complex processes that must be achieved at line speeds, such as sorting packet finishing tags and updating a virtual clock, [8,15–19]. However, the approach taken in this research has been to target the implementation of fair queuing by deriving novel high-speed circuits, rather than settle for alternatives that merely approximate the fairness properties provided by fair queuing.

Due to the line-speeds required for future Internet applications and services, this work has focused exclusively on a hardware implementation in order to gain a performance advantage over existing scheduling implementations, which are typically software based. A WFQ policy has been used to implement the packet scheduler. Although WFQ does not provide quite the same level of fairness as WF²Q, it is less complex to implement. In this case there is a trade-off balancing the complexity of the hardware design and the guaranteed worst case fairness. However by implementing the scheduler in hardware, this research shows the advanced QoS fairness guarantees enabled by fair queuing can be delivered without sacrificing the throughput performance achieved by less complex, less fair, techniques.

3. Decomposition of WFQ traffic scheduler

The design of the WFQ traffic scheduler has been decomposed into a number of separate parts, as illustrated in Fig. 3. The packets follow a path through the circuit from left to right. The packets enter the scheduler input and the required data such as traffic class, flow ID, etc. is parsed before the packet data itself is stored in the shared packet buffer. The WFQ calculation itself is carried out in the WFQ tag computation circuit, which among other functions produces a “finishing tag” for each packet received. These tags are passed on to the finishing tag sort/retrieve circuit, which sorts and stores tags in order, from smallest to largest. A pointer to the location of the associated packet data payload in the shared packet buffer is stored along with the finishing tag values. When a packet is scheduled (according to the WFQ algorithm) to be transmitted at the output of the scheduler, the lowest finishing tag and associated shared buffer pointer are obtained from the tag storage memory. The packet server can then retrieve the correct packet payload from the buffer and it is transmitted. The three main elements in the circuit architecture will now be investigated and derived separately.

3.1. WFQ computation circuit

This circuit in the packet scheduler is responsible for all calculations necessary to operate the WFQ algorithm. It also

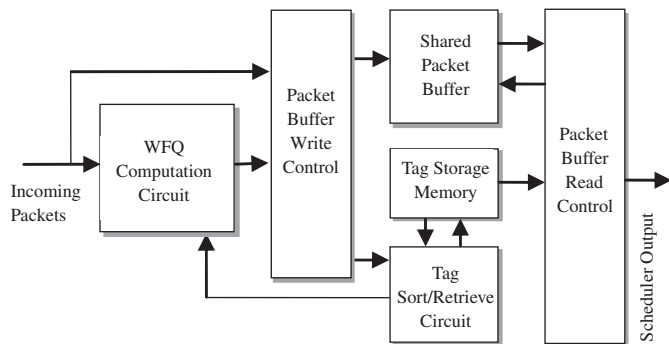


Fig. 3. WFQ packet scheduler architecture overview.

tracks the data associated with active session connections, which is necessary for the operation of the WFQ calculation. Packet information is received at the input along with a pointer to the location of the related packet payload in the shared buffer. At the output, finishing tags are passed to the tag sort/retrieve circuit along with the associated pointer.

To simulate GPS using WFQ, a virtual time measure, $V(t)$, is used. The operation of the scheduler can be separated into two distinct parts, packet arrival and packet departure, each of which is defined as an event, where t_j represents the time at which the j th event occurs. The set of sessions that are busy in an interval (t_{j-1}, t_j) is fixed and denoted as B_j . In WFQ, each session represents a virtual queue. $V(t)$ is set to zero for all times when the server is idle, and ϕ_i is the weight set for session i . For an interval τ with a constant set of backlogged flows within any busy period, $V(t)$ increases using (1):

$$V(t_{j-1} + \tau) = V(t_{j-1}) + \frac{\tau}{\sum_{i \in B_j} \phi_i}, \quad \tau \leq t_j - t_{j-1}, \quad j = 2, 3, \dots \quad (1)$$

$\text{Next}(t)$ is the next point in real time at which the set of backlogged flows may change as a result of a packet departure, thus effecting the slope of $V(t)$. $\text{Next}(t)$ is calculated using (2) assuming that there are no further arrivals of packets in the interval $(t, \text{Next}(t))$. F_{MIN} is the minimum value of finishing tag yet to depart the GPS simulation:

$$\text{Next}(t) = t + (F_{\text{MIN}} - V(t)) \sum_{i \in B_j} \phi_i \quad (2)$$

A start tag is calculated for each packet that arrives, using (3) and once this is set, the finishing tag can be computed using (4):

$$S_i^k = \max\{F_i^{k-1}, V(a_i^k)\} \quad (3)$$

$$F_i^k = S_i^k + \frac{L_i^k}{\phi_i} \quad (4)$$

where S_i^k is the start tag given to the k th packet in the session i ; F_i^k is the finishing tag of the k th packet in the session i ; $V(a_i^k)$ is the virtual time in GPS when the k th packet from session i arrives, L_i^k is the length of the k th packet in the session i .

Fig. 4 shows the modular circuit architecture developed to implement the WFQ algorithm. Three dual port memories are utilized, which are addressed based on each packet's flow ID values, where each flow is represented by a single virtual queue. For each session the memories store the weight, previous finishing tag and the number of packets yet to leave the server. The memories used are 16-bits wide, while the number of supported queues is determined by the size of memory available.

An initial version of the complete finishing tag computation circuit has been synthesized, targeting 130 nm ASIC technology. The circuit operates with clock period of 5.8 ns, or 173 MHz [20]. It deploys 3 pairs of 8-kByte dual-port memory blocks, using

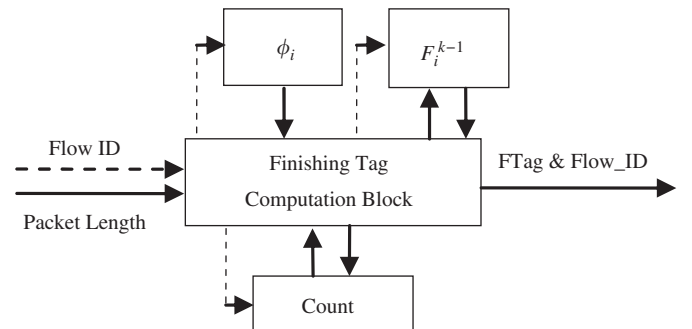


Fig. 4. Finishing tag block level description.

13-bit flow ID values. This enables the circuit to support up to 8000 different service classes (virtual queues), each with its own individual QoS parameters. The circuit combines a number of functional blocks, along with distributed memories in a fully pipelined architecture, to allow the data path to compute one finishing tag per clock cycle. The number of flows in the circuit is limited by the memory size to 8000 flows. However, 16-bit addressing is accommodated, therefore if additional memory is available, the same circuit can handle 64,000 independent flows.

3.2. Finishing tag sort/retrieve circuit

This is a time critical component for most fair queuing algorithms and represents a key bottleneck, particularly for the software implementations common in industry. Although tags can be sorted using established software algorithms [21], no satisfactory design exists to facilitate the process effectively in hardware at high speed. For example, the classic van Emde Boas method is unsuitable for implementation in hardware [22]. A key research aim is to bridge the gap between the accuracy of software and the potential throughput achieved by hardware.

Most research in this field implements the tag sorting mechanism using queue/heap methods in software. These are generally limited to $O(\log N)$ performance. Various types of calendar queues have been implemented [23,24]; however, it has been shown that these are limited in size and scalability. A two dimensional calendar queue (TCQ) [25] claims $O(\log \log N)$ performance, but it degrades the delay guarantees provided by the WFQ algorithm. The Leap Forward Virtual Clock (LFVC) algorithm has the same performance as TCQ but also similar drawbacks affecting the accuracy of the implementation of the scheduling algorithm [26].

Stratified Round Robin (SRR) is an approach suitable for hardware implementation that uses “finite universe priority queues” to sort packets among tens of classes [22]. However, as shown in Section 2, round robin is inherently less fair than fair queuing, and furthermore, in SRR the number of traffic classes is very limited. The “binning” technique, developed for a Credit Based Fair Queuing (CBFQ) hardware implementation [12], is also unsatisfactory because it aggregates values together in groups introducing inaccuracy.

A prime reason given for developing SRR was the bottleneck of sorting tags, which emphasizes that tag sorting is often seen as an obstacle against using fair queuing. The tag sorting circuit developed in this research overcomes this obstacle.

3.2.1. Optimized solution for hardware

A comparison of the properties of a range of search and sort methods was performed [27]. The analysis included; calendar queue, Fibonacci heap, binomial heap, LFVC, binning (value aggregation), binary search, CAM, TCAM, tree and multi-bit tree. Consideration was made in terms of the number of memory accesses per operation, memory access time requirements, accuracy due to search granularity/aggregation of data, value insertion, value removal/deletion update times, worst case performance, and hardware timing constraints. A major factor limiting a number of potential solutions was that they could not deliver the smallest value from a set of tags within a fixed and predictable time period. This is vital for optimizing performance in hardware. The other modules in the scheduler operate with fixed pipelined timing strategies. If the tag sort/retrieve circuit operation is non-deterministic, integrating it into the broader scheduler circuit becomes complex, and importantly, the overall throughput will not be optimal and the advantages of using hardware would be reduced.

Results of the study show that a multi-bit tree offers a fixed guaranteed sort time, comparing favorably against existing standard software algorithms in this regard. The multi-bit tree does not compromise accuracy by aggregating data, as in some of the other hardware solutions previously described. Furthermore, the multi-bit tree is well suited for implementation using multiple distributed memories to enable high-speed parallel processing in hardware.

3.2.2. Tag sort/retrieve circuit architecture

A custom tag sort/retrieve circuit based on a multi-bit tree has been derived where the sorting and storage elements have been separated such that they are independently scalable and configurable, allowing different parameters of memory size and search granularity to be realized. The circuit architecture consists of a number of distributed memory elements accessed by a set of custom designed logic circuits that enable data lookup, as illustrated in Fig. 5.

The full derivation and operation of the circuit architecture is available in [27]. A sorting mechanism and memory structure utilizing a multi-bit tree stores whether a finishing tag value is present in the tag storage memory. For incoming tags, a search for a matching tag value in the tree is used to place the new tag beside its closest match in the tag storage memory, a process which includes setting a tag marker in a translation table. Separating the search function from the data storage, via the translation table, allows the lookup function to be implemented very efficiently in hardware since memory pointers do not have to be accommodated directly alongside the circuitry used for the search.

The tag storage memory uses a linked-list structure to store the tag entries in order, which means that the smallest tag value, i.e. the tag to be serviced next, is always known and instantly accessible. It also enables tags to be deleted from the end of the list as they depart. This removes a processing bottleneck that exists with heaps and queues.

The initial architecture was developed and implemented with a 3 level multi-bit tree, handling 12-bit words, where literals of 4-bits are represented in each level by 16-bit nodes. The first two levels of the tree are relatively small, 272-bits in total, so these are implemented using registers. The third level is 4kbits and is implemented using single port on-chip SRAM. This relatively small amount of on-chip memory allows very fast access to the data needed to operate the search function. The size of memory, in bits, required for each level of the tree (level memory, LM) is

$$LM = 2^{(\log_2(b)) \times l} \quad (5)$$

And it follows that the total memory required for the tree, M , can be expressed as

$$M = \sum_{i=1}^k 2^{(\log_2(b)) \times i} \quad (6)$$

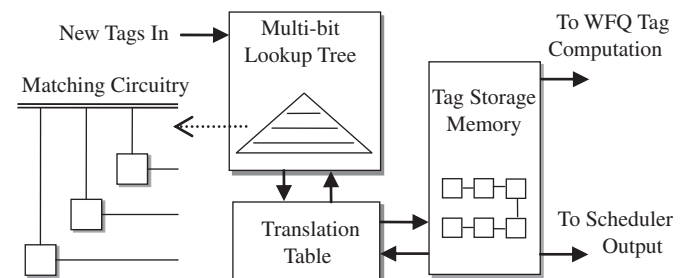


Fig. 5. Finishing tag sort/retrieve circuit architecture.

where l is the level number (level 1 is the tree root), b the branching factor; and k the total number of levels.

Therefore, using a multi-bit tree rather than a binary tree allows the search operation to be accelerated as well as requiring less memory.

At each level of the tree custom circuits are required to perform a matching function comparing the input literal with the existing node value. If an exact match is found it means a value in the tree already exists with this 4-bit literal at this level, consequently the search will continue in the next level. If a non-exact match occurs in any level, i.e. only a smaller value than that requested is present, then all subsequent levels return their maximum value.

A separate detailed investigation of custom matching circuit designs examined look ahead based circuits, including a simple ripple cell approach, a standard look-ahead circuit as well as block look-ahead, skip & look-ahead, and select & look-ahead circuits. All circuits are based on modified adder carry chain acceleration techniques. Of the five accelerated matching techniques, a select & look-ahead approach was the fastest and most hardware efficient option available. The 16-bit version has been used in the prototype circuit, which uses 4-bit literals; however the design is also sufficiently fast to allow the tree nodes to be implemented with 32 bits, accommodating 5-bit literals and 15-bit words. The translation table would expand to 32k entries as a result, and the granularity of search possible would be increased [28].

3.2.3. Tag storage memory

A system of linked-lists is used to implement the tag storage memory. Each entry in the tag storage list stores a tag value and a pointer to the next link, which will be the next biggest tag value in the memory. The link at the head of the list is the smallest tag value. The list is implemented in this case using FPGA block RAM resources. The tag storage memory and the tree are independently scalable and configurable—the granularity or accuracy of sorting the tags depends on the tag sort/retrieve circuit, while the size (word width) and number of tags stored is decided by the size of RAM available for tag storage.

The translation table provides an essential bridge between the search tree and the tag storage memory allowing them to be separately scalable. The translation table records the physical memory address of each tag value in the linked list, where the position of the record in the table is addressed using the tag value itself.

Depending on the accuracy of the WFQ computation, tag values may be rounded off so that theoretically two or more tags of the same value can exist in the scheduler at one time; however, the sequential nature of the linked list allows a first-come-first-served policy to be applied when duplicate values exist. When adding a new tag value that already exists in the system, the translation table records only the most recent tag to have entered the linked list, while duplicate tags are inserted beside each other in the linked-list, and are served in the order received by the tag lookup circuit.

The tag storage memory requires 4 clock cycles (2 read and 2 write cycles) to complete a full insertion cycle, while the tree and translation table require a total of 4 clock cycles to throughput one tag. This arrangement therefore allows the operation of these components to be optimally synchronized and efficiently pipelined. When implemented using UMC 130 nm standard cell technology, the circuit is capable of supporting sort/retrieve operations at a clock frequency of 143 MHz, which equates to 35.8 million sort/retrieve operations per second, or 35.8 million packets per second [28].

3.3. Shared packet buffer

The final component of the WFQ packet scheduler is the shared packet buffer. When a new packet enters the server, its data payload is sent to the packet buffer. The buffer produces a memory pointer for the location of the packet data, which is forwarded to the finishing tag lookup table. When a packet is served at the scheduler output, a pointer is retrieved from the lookup table and used to locate the correct packet payload in the buffer, which reads this data to the scheduler output.

Studies have identified the shared buffer as the optimum buffering technique [29], because it utilizes memory more efficiently and has a lower packet loss rate for a given buffer space, compared to strategies such as the input buffer [30] and output buffer [31]. Shared buffer architectures already exist for ATM switches [32] using 32 and 16-byte cells, but these cannot buffer variable sized packets as required in IP networks. However, recent work has shown that a shared buffer targeting variable size packets is possible using standard FPGA technology [33].

3.3.1. Shared buffer implementation

For functions with high data capacity requirements such as the shared packet buffer, memory access is frequently a limiting factor when FPGAs are used. SRAM memories perform well in terms of latency but their high cost and low memory capacity make them impractical for high capacity buffering in network processing. DDRAM II memory architectures have high memory capacities but are hindered by latency. Consequently, RLDRAM II memory has been used, which is a type of memory with a capacity similar to that of DDRAM II, but with a lower latency. It employs a double data rate I/O for increased bandwidth and an eight-bank architecture that decreases the probability of random access conflicts and is optimized for high-speed operation.

There are separate I/O (SIO) and common I/O (CIO) option modes. SIO has separate READ and WRITE ports to eliminate bus turnaround cycle (switching between read and write) and contention, thus achieving full bus utilization. CIO has a shared READ/WRITE port that requires additional cycles in order to switch the memory bus between read and write access. It is optimized for data streaming, where the near-term bus operation is either 100% READ or 100% WRITE. For the shared buffer implementation, the memory bandwidth must be fairly divided into a read and write access when the RLDRAM access operates in CIO mode. Switching the common I/O between read and write access costs an additional time, called bus-turnaround-time. Therefore, a no-operation (NOP) cycle-slot must be included. The access latency to a specific memory bank is determined by the row-cycle times, and read or write latencies.

Previous work [34] has shown that CIO mode can be utilized within a shared buffer to support 12.8 Gb/s throughput. This is scalable up to 20 Gb/s if SIO mode is utilized, but at the penalty of higher hardware costs. The work focused on optimizing the memory bandwidth achievable for different payload sizes, assuming a minimum payload of 60-bytes with a 20-byte header. The packet buffer controller targeted the same Altera Stratix II speed grade—4 device used onboard the Altera Traffic Management board, which has been used for the overall WFQ scheduler implementation. The RLDRAM II required the controller to operate at 200 MHz which is easily attainable using Stratix II FPGA technology. The research showed a TDM cadence of 4-2-4-2 was effective at optimizing bus turnaround times for optimum memory bandwidth at 200 MHz. The shared packet buffer used 6668 ALUTs (less than 8% of the device).

Fig. 6 illustrates the implemented architecture. The data for individual packets is divided between cells of size 128-bytes.

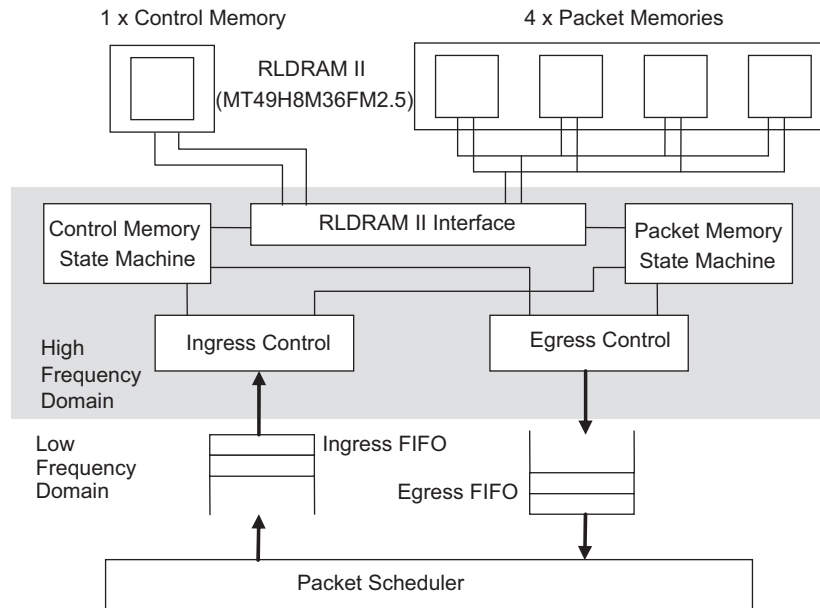


Fig. 6. Shared packet buffer architecture.

A linked-list is used to track the position of subsequent cells belonging to individual packets, which are distributed throughout the shared buffer memory.

The architecture consists of five CIO RLDII memory chips, four are 8-Mbyte devices with 36-bit data width and one is a 16-Mbyte device with an 18-bit data width. The four 8-Mbyte devices store the packets and the single 16-Mbyte device stores a linked list (queue descriptors). The memory interface operates at a 200 MHz, high frequency domain, while the shared buffer control circuit operates at 100 MHz, low frequency domain. The dual data rate allows memory access at both the rising and falling edges of the clock. FPGAs are well suited to multiple clock domains, since they already have a number of clock trees built into the fabric of the device. Dual clock FIFOs have been implemented, enabling data-access synchronization between the two clock domains.

4. Full WFQ scheduler FPGA implementation and synthesis results

The full WFQ traffic management scheduler comprising the WFQ algorithm computation circuit, tag sort/retrieve circuit, and the shared packet buffer has been implemented using the Altera Traffic Management development board, which utilizes Stratix II FPGAs, and accesses onboard resources such as RLDII memory components for the shared buffer and QDRII memories for use with the control logic. In the implementation the system clock operates at 100 MHz while the memories operate at 200 MHz, as previously discussed in the description of the shared buffer. The full architecture that has been implemented is shown in Fig. 7.

The synthesized circuit is capable of supporting WFQ traffic scheduling at line-speeds of 12.8 Gb/s, conservatively assuming an average packet sizes of 80-bytes. Table 1 shows the resource utilization following synthesis targeted at the Altera EP2S130F1508C4 FPGA device. Approximately 17% of the available logic resources are used for implementing the WFQ circuit, with sufficient logic resources remaining to implement different network processing functions, such as packet header processing, flow classification and management on a single device.

The overall system bandwidth at the control device (FPGA), including ingress egress traffic, RLDII access bandwidth (packet storage) and QDRII access bandwidth (WFQ descriptors) is approximately 100 Gb/s. This is at the upper limit of the I/O bandwidth capabilities of the FPGA device used for this prototype, which is the constraining factor for implementing extremely high-throughput WFQ circuits beyond 20 Gb/s.

4.1. Testing and verification

In order to verify the operation of the WFQ packet scheduler, two performance benchmark tests were carried out. In the first case the scheduler has been tested by injecting traffic flows into the circuit and examining the output. Three flows A, B and C, were each allocated different weights, so that according to the WFQ algorithm they are guaranteed 60%, 30% and 10%, respectively, of the available bandwidth.

With the scheduler disabled, the traffic flows were injected into the device and a backlog of traffic was allowed to build up. The flows were injected with a ratio of 60:30:40 (A:B:C), which means that flow C is generating more traffic than is allocated under the WFQ weights specified above—it is acting like a “badly behaved” flow.

When the scheduler was enabled, the results showed that each flow was consequently allocated the correct amount of bandwidth specified by the WFQ weights applied, i.e. 60%, 30% and 10%. Fig. 8 shows how the flows are allocated the correct bandwidth. Note how the packets from flow C, which is badly behaved, are only given 10% of the available bandwidth, while the packets from the other flows are given priority. Only when flows A and B are not sending traffic are the backlogged packets of flow C scheduled for service.

The figures from the experimental results are shown in Table 2. Note the results cover the period while all three flows are backlogged.

The second test involved creating a model of the operation of the ideal WFQ algorithm using OPNET to simulate the expected behavior. The expected behavior from this WFQ simulation could then be compared against a behavioral implemented hardware. This allowed the validity of the hardware implementation to be examined in terms of fairness and throughput guarantee.

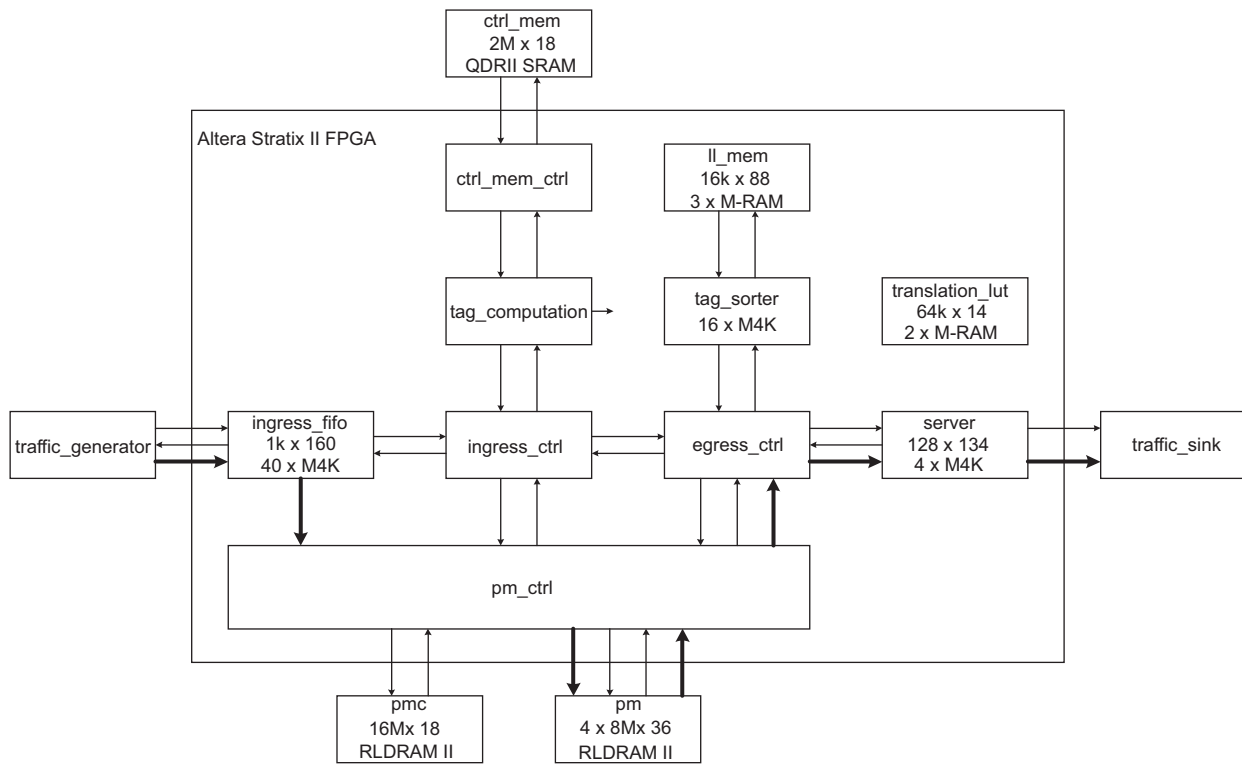


Fig. 7. Full WFQ packet scheduler architecture.

Table 1
Post-layout synthesis results.

Resource	Used/Total
System clock	100 MHz
Memory clocks	200 MHz
ALUTs	18,542/106,032 (17%)
Pins	410/1127 (36%)
PLLs	4/12 (33%)
M-RAMs	5/6 (83%)
M4Ks	68/609 (11%)

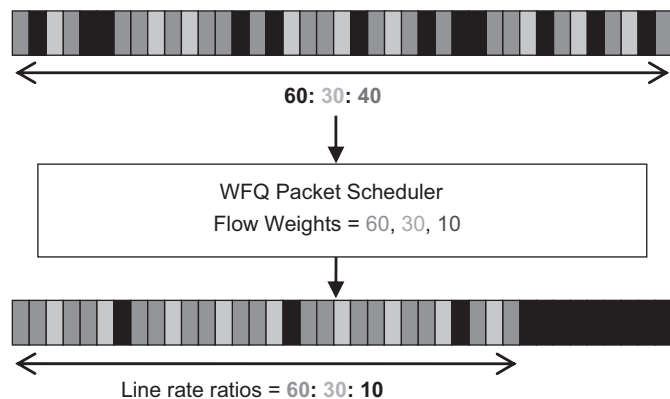


Fig. 8. Packet by packet throughput behavior results when a badly behaved flow is processed through the WFQ packet scheduler.

Table 2
Throughput allocation experimental figures.

Flow	Incoming line rate	Bytes served	Served line rate
1	46.15%	20,985	59.97%
2	23.08%	10,512	30.04%
3	30.77%	3497	9.99%

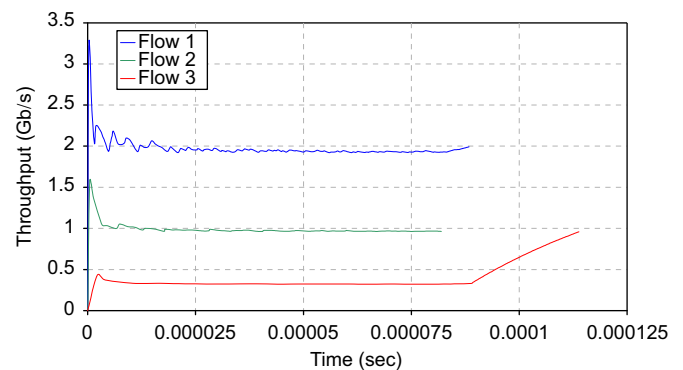


Fig. 9. Throughput provided by ideal WFQ simulated in OPNET.

flows has also shown a bandwidth allocation error tolerance of less than 0.1%.

5. Conclusion

To the knowledge of the authors, the WFQ packet scheduler presented in this paper is the first full hardware implementation of such an architecture to be published. The proposed circuit architecture provides fine granularity QoS support for Internet traffic at multi-gigabit throughput rates. Both the granularity of

Figs. 9 and 10 show the throughput performance for the ideal WFQ model and the hardware model simulated in OPNET. These results show a close correlation between the ideal model and the hardware design. Furthermore, Fig. 11 shows actual traffic measured from the circuit using a traffic analyzer under the same test conditions. Analysis of the circuit's performance with test packet

QoS and the throughput rates supported are not achievable with previous state of the art systems. The circuit is therefore ideal for deployment with high bandwidth wireline and wireless technologies, such as LTE, where tiered services based on SLA's are emerging for users and content providers.

The fully functional 12.8 Gb/s FPGA based prototype circuit demonstrates that complex packet scheduling algorithms can be implemented in hardware to achieve accurate QoS provision, while delivering low latency and high throughput performance. The flexible, modular design approach supports the scalability of the architecture to use standard cell ASIC technology, suitable for 50 Gb/s line speeds.

The WFQ computation circuit is able to process at speeds exceeding the performance of typical industrial software based implementations. Support for such high numbers of sessions at the high throughput rates achieved is only possible using dedicated hardware. The presented implementation is dimensioned to support 64,000 sessions. The number of supported sessions (or individual IP flows) can be further scaled well beyond one million by increasing the finishing tag lookup table hierarchy.

A high throughput finishing tag sort/retrieve circuit has also been derived that can support tag (or time-stamp) sorting at multi-gigabit line-speeds. This is a key contribution to resolving the issue of a much referenced bottleneck that is typically encountered when implementing fair queuing algorithms [15–19]. The authors suggested that the circuit might also be used to support other fair queuing algorithms other than WFQ, such as WF²Q.

Finally an investigation into a shared packet buffer memory architecture has shown that by using RLDRAM II technology, with an optimized TDM cadence strategy, the buffer can support a throughput rate of 12.8 Gb/s for an average packet size of 80-bytes. Due the performance constraints of the technology used, in particular the memory bandwidth constraints, it is the shared packet buffer that limits the performance of this implementation of the packet scheduler architecture.

In order to address this bottleneck, further work has already been completed that utilizes a new circuit architecture for a shared buffer memory which has been implemented using Altera Stratix III FPGA with DDR3 memory modules. This circuit is suitable for supporting data throughput rates of up to 50 Gb/s [35]. Consequently, the authors propose that the throughput performance of the WFQ computation and sort/retrieve circuits can equally be improved by utilizing the same advanced silicon and memory technologies, so that the complete packet scheduler circuit can fully support 50 Gb/s.

The high throughput and low latency of the system is achieved by means of parallel dedicated processing units to deal with computation of scheduling algorithms, virtual queue management, and the service of the packets in accordance with the scheduling policy. However, this improved performance does not come for free, and the obvious disadvantage is that the architecture increases the hardware cost and the amount of distributed memory necessary. From a hardware cost perspective, the use of replicated hardware logic for queue management and dedicated distributed memory to hold the virtual queue descriptors adds a significant cost in comparison to traditional software based

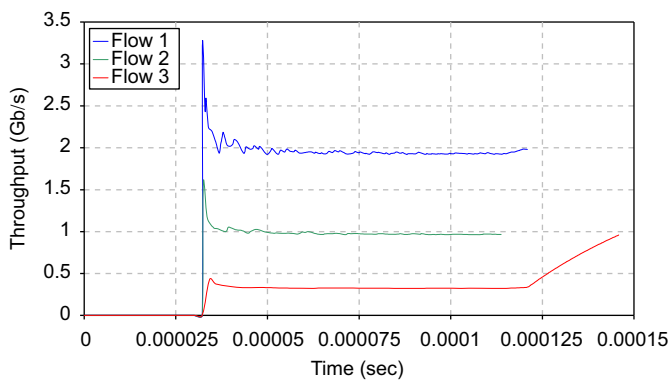


Fig. 10. Throughput provided with hardware model simulated in OPNET.

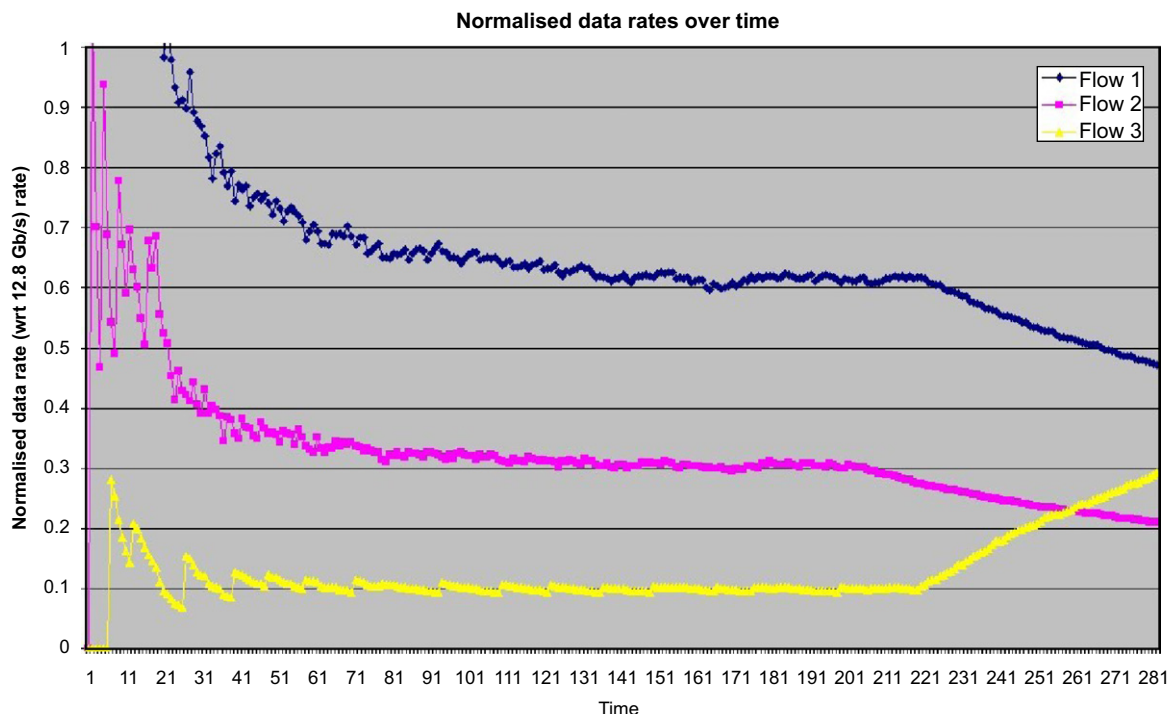


Fig. 11. Normalized measured traffic throughput of circuit over time.

architectures, or even low performance hardware based alternatives. Nevertheless, this is a very familiar trade-off for vendors in the network processor industry. Consequently, processes where high throughput performance is essential are commonly offloaded to hardware accelerators.

The proposed WFQ circuit architecture and the research work presented in this paper has demonstrated and proven that high-performance and highly accurate WFQ implementations can be achieved purely in hardware. Furthermore, the modular approach of the proposed architecture permits the scalability of the WFQ circuits beyond 50 Gb/s, using state of the art semiconductor and memory technologies, providing sustainable throughput performance for emerging QoS sensitive high-bandwidth Internet applications in the future.

Acknowledgments

The authors would like to thank Altera Corporation for their financial and technical support. This work was also supported by the Engineering and Physical Sciences Research Council, under grant EP/E028640/1.

References

- [1] I. Siomina, S. Wanstedt, The impact Of QoS support on the end user satisfaction in LTE networks with mixed traffic, in: Proceedings of the IEEE 19th International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC 2008, September 2008.
- [2] M. Anas, C. Rosa, F.D. Calabrese, K.I. Pedersen, P.E. Mogensen, Combined admission control and scheduling for QoS differentiation in LTE uplink, in: Proceedings of the Vehicular Technology Conference, 2008. VTC 2008-Fall. IEEE 68th, September 2008, pp. 1–5.
- [3] J. Sommers, P. Barford, N. Duffield, A. Ron, Multi-objective monitoring for SLA compliance, *IEEE/ACM Transactions on Networking* 18 (2) (2010) 652–665.
- [4] S. Chen, K. Nahrstedt, An overview of quality of service routing for the next generation high-speed networks: problems and solutions, *IEEE Network Magazine*, Special Issue on Transmission and Distribution of Digital Video (1998).
- [5] S. Gupta, M. Zarki, Traffic classification on scheduling in ATM networks, *Telecommunication Systems Modelling Analysis Design and Management* 2 (Part 1) (1993) 51–69.
- [6] M. Shreedhar, G. Varghese, Efficient fair queuing using deficit round robin, in: Proceedings of the ACM SIGCOMM, September 1995, pp. 231–242.
- [7] A. Demers, S. Keshav, S. Shenker, Analysis and Simulation of a fair queuing algorithm, in: Proceedings of the ACM SIGCOMM'89, 1989, pp. 3–12.
- [8] H. Chaskar, U. Madhow, Fair scheduling with tunable latency: a round-robin approach, *IEEE Transactions on Networking* 11 (4) (2003) 592–601.
- [9] A. Parekh, R. Gallager, A generalized processor sharing approach to flow control in integrated services networks: the single node case, *ACM/IEEE Transactions on Networking* 1 (1993) 344–357.
- [10] Dimitrios Stiliadis, Anujan Varma, Efficient fair-queuing algorithms for packet-switched networks, *IEEE/ACM Transactions on Computers* 6 (2) (1998) 175–185.
- [11] S. Golestani, A self-clocking fair queuing scheme for broadband applications, in: Proceedings of the INFOCOM'94, 1994, pp. 636–646.
- [12] B. Bensaou, D. Tsang, K. Chan, Credit based fair queuing (CBFQ): a simple service scheduling algorithm for packet switched network, *IEEE Transactions on Networking* 9 (5) (2001) 591–604.
- [13] J. C. R. Bennett, H. Zhang, WF²Q: worst-case fair weighted fair queuing, in: Proceedings of the INFOCOM'96, 1996, pp. 120–128.
- [14] J.C.R. Bennett, H. Zhang, Hierarchical packet fair queuing algorithms, *IEEE Transactions on Networking* 5 (1997) 675–689.
- [15] Rong Pan, Lee Breslau, Balaji Prabhakar, Scott Shenker, Approximate fairness through differential dropping, *SIGCOMM Computer Communication and Review* 33 (2) (2003) 23–39.
- [16] V. Ramaswamy, L. Cuellar, S. Eidenbenz, N. Hengartner, Preventing bandwidth abuse at the router through sending rate estimate-based active queue management, in: Proceedings of the IEEE International Conference on Communications, 2007, pp. 569–574.
- [17] G.N. Rouskas, Z. Dwekat, A practical and efficient implementation of WF2Q+, in: Proceedings of the IEEE International Conference on Communications, ICC '07, 2007, pp. 172–176.
- [18] Dong-Hoon Yi, Jong Won Kim, A frame-based packet scheduling scheme for real-time applications: 2-tier hierarchical frame queueing, in: Proceedings of the CCNC 2008—5th IEEE Consumer Communications and Networking Conference, vol. 5(1), January 2008, pp. 238–242.
- [19] Chakchai So-In, Raj Jain, Abdel-Karim Tamimi, Scheduling in IEEE 802.16e mobile WiMAX networks: key issues and a survey, *IEEE Journal on Selected Areas in Communications* 27 (2) 156–171.
- [20] C. McKillen, S. Sezer, X. Yang, High performance service-time-stamp computation for WFQ IP packet scheduling, in: Proceedings of the IEEE Computer Society Annual Symposium on Emerging VLSI Technologies and Architectures, 2006.
- [21] P. van Eerde Boas, R. Kaas, E. Zijlstra, Design and implementation of an efficient priority queue, *Mathematical Systems Theory* 10 (1977) 99–127.
- [22] S. Ramabhadran, J. Pasquale, The stratified round robin scheduler: design, analysis and implementation, *IEEE/ACM Transactions on Networking* 14 (2006).
- [23] J.L. Rexford, A.G. Greenberg, F.G. Bonomi, Hardware-efficient fair queueing architecture for high-speed networks, in: Proceedings of the INFOCOM, March 1996.
- [24] D.C. Stephens, J.C.R. Bennett, H. Zhang, Implementing scheduling algorithms in high speed networks, *IEEE JSAC* (1999).
- [25] A. Francini, F. Chiussi, A weighted fair queueing scheduler with decoupled bandwidth and delay guarantees for the support of voice traffic, in: Proceedings of the GLOBECOM '01. IEEE, vol. 3, 2001.
- [26] S. Suri, G. Varghese, G. Chandramenon, Leap forward virtual clock: a new fair queueing scheme with guaranteed delays and throughput fairness, in: Proceedings of the INFOCOM'97, 1997.
- [27] K. McLaughlin, S. Sezer, H. Blume, X. Yang, F. Kupzog, T. Noll, A scalable packet sorting circuit for high-speed WFQ packet scheduling, *IEEE Transactions on VLSI Systems* 16 (7) (2008) 781–791.
- [28] F. Kupzog, K. McLaughlin, S. Sezer, H. Blume, T. Noll, J. McCanny, Design and analysis of matching circuit architectures for a closest match lookup, *IEEE AICT*, Guadeloupe, February 2006.
- [29] H. Kuwahara, A shared buffer memory switch for an ATM exchange, *IEEE GLOBECOM* (1989).
- [30] C. Minkenberg, T. Engbersen, A combined input and output queued packet switched system based on PRIZMA switch on a chip technology, *IEEE Communication Magazine* (2000).
- [31] M. Hluchyj, M. Karol, Queuing in high-performance packet switching, *IEEE Journal, SAC-5* (1987).
- [32] J.W. Shim, G.J. Jeong, M.K. Lee, FPGA implementation of a scalable shared buffer ATM switch, *ATM*, June 1998.
- [33] S. O'Kane, C. Toal, S. Sezer, Design and implementation of a shared buffer architecture for a gigabit ethernet packet switch, in: Proceedings of the IEEE SOCC 2005, pp. 283–286.
- [34] D. Burns, C. Toal, K. McLaughlin, S. Sezer, M. Hutton, K. Cackovic, An FPGA based memory efficient shared buffer implementation, *Field Programmable Logic and Applications, FPL* (2007) 661–664.
- [35] X. Yang, S. Sezer, J. McCanny, D. Burns, DDR3 based lookup circuit for high-performance network processing, in: Proceedings of the IEEE SOCC2009, September 2009, pp. 351–354.

Kieran McLaughlin received his ME degree in Electrical & Electronic Engineering from Queen's University Belfast, UK, in 2003. He received his Ph.D. from Queen's University in 2006, where his research focussed on high-speed lookup circuit architectures for network processing, which generated a number of related patents. He joined the SoC Network Processing research group at ECIT in 2006. He is currently a Senior Engineer at the Centre for Secure Information Technology (CSIT) at Queen's University, where his research interests focus on high-throughput technologies for network security processing. He has published over 20 conference and journal papers.

Dwayne Burns is a Principal Engineer at ECIT, Queen's University Belfast, where he has led the development of several technology demonstrators, including a weighted fair queuing traffic manager, and a high performance regular expression-based content processor. Previously, he has worked in both start-up and multi-national companies, at the management and engineering levels. He holds two patents and has published over forty papers in the fields of network processing, liquid crystal on silicon spatial light modulators, optoelectronic neural networks and polymer on LED microdisplays. He holds a Ph.D. in Optoelectronics (1994) and a first class honours degree in Physics & Electronics (1991), both from the University of Edinburgh, UK.

Ciaran Toal received his ME, M.Sc. and D.Phil degrees in Electrical and Electronic Engineering from Queen's University Belfast in 2001, 2002 and 2005, respectively. He is an engineer at the Institute of Electronics, Communications and Information Technology, Queen's University, Belfast. His research interests include network processing architectures and high-speed wireless technologies. He has built custom hardware architectures for network processing applications such as memory controllers, run-time programmable CRC circuits. He has implemented an entire 1 Gb/s 4x4 wireless MIMO transceiver on an FPGA. He has published more than 20 papers and a number of technology patents.

Colm McKillen graduated with an ME degree in Electrical and Electronic Engineering from Queen's University of Belfast in 2003. He began his Ph.D. within

the System-on-Chip group at ECIT in the same year. He is currently investigating high speed programmable scheduling architectures for custom hardware.

Sakir Sezer received his Dipl. Ing. in Electrical and Electronic Engineering from RWTH Aachen University, Germany, and his Ph.D. in 1999 from Queen's University, Belfast, UK. He is currently the SoC Research Director and the Head of Network Security Research in the School of EECS at Queen's University, Belfast. His research

is leading major (patented) advances in the field of high-performance content processing and is currently commercialised by Titan IC Systems. He has co-authored over 120 conference and journal papers in the area of high-performance network, content processing and System on Chip. He has been awarded a number of prestigious awards including InvestNI, Enterprise Ireland and Intertrade Ireland innovation and enterprise awards and the InvestNI Enterprise Fellowship. He is also a co-founder and CTO of Titan IC Systems and a member of the IEEE International System-on-Chip Conference executive committee.